

01:Build AR4 simulator MVP

AR4 MK5 網頁機器人模擬器 - 開發 Session 完整摘要

本文件記錄從官方 STL 與 ROS 2 URDF 建立 AR4 MK5 六軸機械結構、完成 Three.js 數位分身、關節控制、Forward Kinematics、Inverse Kinematics、TCP 與標準視圖控制的完整 MVP 實作。

項目	目前狀態
Repository	https://github.com/sabgkang/AR4-simulator
Branch	main
最新 Commit	f934cc8 - Refine robot calibration and coordinate views
本機入口	http://localhost:3000/
技術	Node.js、React、TypeScript、Three.js、Vinx / Vite

文件標題保留使用者指定的冒號；Windows 實際檔名因系統限制改用連字號。

1. 專案與技術架構

- 以 Node.js、React 19、TypeScript 與 Three.js 建立互動式六軸機器人模擬器。
- 使用 STLLoader 載入官方 AR4 MK5 STL，OrbitControls 提供旋轉、平移與縮放。
- Vinext / Vite 負責開發與正式建置；主畫面由單一 client component 管理 Three.js 場景與控制狀態。
- 主要程式為 app/robot-simulator.tsx，版面樣式為 app/globals.css。

2. AR4 MK5 STL 模型

專案模型資料夾保留 22 個 STL；目前 MVP 實際載入 18 個可視零件。四個 Link_1_Col.STL 至 Link_4_Col.STL 保留在來源資料夾，但未加入目前場景。

部位	執行時載入的 STL
Base	Aluminum、Enclosure、Motor
Link 1	Aluminum、Motor
Link 2	Aluminum、Motor、Cover、Logo
Link 3	Aluminum、Motor
Link 4	Aluminum、Motor、Cover、Logo
Link 5	Aluminum、Motor
Link 6	Aluminum

3. 機器人關節控制

六個關節均保留滑桿並加入鍵盤數字輸入；輸入欄可顯示 -155.25，支援 0.01 度精度、Enter 提交與 Escape 還原。

關節	名稱	範圍
J1	Base	-170° 至 +170°
J2	Shoulder	-42° 至 +90°
J3	Elbow	-89° 至 +52°
J4	Wrist roll	-165° 至 +165°
J5	Wrist bend	-105° 至 +105°
J6	Tool roll	-155° 至 +155°

J1 正方向已校正：從 +Z 上視時，正角度依數學定義逆時針增加。

4. 關節零點與安裝幾何

校正過程中曾修改 J1、J2、J3 的零點，後來撤回舊校正並重新建立正確的機械 frame。最終只有 J1 保留 +90° 內部模型補償。

```
JOINT_ZERO_OFFSETS = [Math.PI / 2, 0, 0, 0, 0, 0]
```

- J2 Shoulder 改為安裝在指定的肩部支架平面，不再錯誤掛載於圓形馬達端蓋。
- 幾何安裝錯誤透過 joint frame 與 visual transform 修正，不用零點偏移掩蓋。

- 每個關節角度在送入 Three.js 前會加上對應的零點補償。

5. 預設姿態

姿態	J1、J2、J3、J4、J5、J6
Home	0, 0, 0, 0, 0, 0
Upright	0, 0, -89, 0, 0, 0
Inspect	0, 45, 20, 0, 0, 0

Home pose 與 Zero all 均回到 Home。

6. Forward Kinematics 與 TCP

- 底部 X / Y / Z 顯示 TCP 紅球中心的世界座標，單位為毫米。
- 紅球與座標數值從同一個 Three.js world transform 取得，避免模型顯示與計算分離。
- TOOL_TIP_OFFSET 已從假設的 55 mm 改為 0.0，因此 TCP 位於 Link 6 參考原點。
- TCP 紅球半徑為 10 mm，跟隨所有上游關節的位置及姿態。

7. TCP 座標軸

設定	數值
X / Y / Z 顏色	紅 / 綠 / 藍
箭頭長度	50 mm
箭頭厚度	5 mm
座標框架旋轉	繞局部 Z 軸 -90°

Forward Kinematics、Inverse Kinematics、TCP 紅球與 TCP 軸均共用相同的世界位置及 quaternion。

8. Base 世界座標軸

Base 世界座標框架固定在世界原點 (0, 0, 0)，X / Y / Z 分別為紅、綠、藍，箭頭長 100 mm、厚 5 mm，不跟隨關節旋轉。

9. Inverse Kinematics

- Cartesian Target 提供 X、Y、Z、 θ_x 、 θ_y 、 θ_z 六個輸入。
- Use current 可擷取目前 TCP pose；Calculate & move 求解並更新六個關節。
- 數值 IK 使用多個初始姿態與有限差分 Jacobian，並限制每一步及所有官方關節範圍。
- 成功時顯示位置及姿態誤差；無解時在面板內顯示訊息，不開 popup dialog。

10. 頁面版面

- 主要區域包含左側導覽列、3D Model View、Manual Control 與 Cartesian Target / IK。
- 各主要欄位使用完整視窗高度，控制區可獨立捲動。

- 高解析度螢幕的最小可讀文字提高至 16 px，包括 mm、deg、範圍與提示文字。
- Joint angle 輸入欄加寬並保留滑桿。

11. 右下角標準視圖控制

點擊平面	相機方向	Orbit target
XY	從 +Z 看向 Base	(0, 0, 0)
XZ	從 +Y 看向 Base	(0, 0, 0)
YZ	從 +X 看向 Base	(0, 0, 0)

- 指示器重畫為外部 SVG，顯示尺寸為 200 x 183 px。
- 三個平面預設為低彩度半透明白色，hover / focus 時分別顯示黃、紫、青色。
- 移除外圍虛線與中央原點圓圈。
- 軸線縮短至箭頭底端，避免線條覆蓋箭頭尖端。

12. Reset View

Reset view 會恢復初始相機位置、OrbitControls target 與 Z-up orientation，避免切換標準平面後保留錯誤的 camera up direction。

13. 開發錯誤處理

Object.send: send was called before connect

此訊息來自 Vite HMR 開發客戶端：WebSocket 尚未完成連線就嘗試傳送訊息，並非機器人運動或 IK 程式的 Object.send。

ResizeObserver loop completed with undelivered notifications

這是版面連續改變尺寸時可能出現的瀏覽器非致命警告。已加入 window.onerror、unhandledrejection 與 ResizeObserver 過濾，避免開發 overlay 持續出現。

瀏覽器錯誤會送至 /_client-log，並寫入 logs/browser-errors.log。

14. Git 與 GitHub

Commit	內容
9d6dacc	Initial AR4 MK5 simulator
f083270	Add inverse kinematics and TCP visualization
39272a7	Calibrate joint frames and TCP orientation
f934cc8	Refine robot calibration and coordinate views

Repository： <https://github.com/sabqkang/AR4-simulator>

目前 main、HEAD 與 origin/main 已同步；最後一次檢查時工作目錄乾淨且正式建置成功。

15. 從 ROS 2 URDF 建立機械結構

本模擬器不是在瀏覽器執行時直接解析 URDF，而是先從官方 ROS 2 MK5 URDF

擷取機械結構，再將必要資料轉寫成 Three.js 階層。這樣能保留 URDF kinematic chain，同時直接使用官方 3D Print STL。

15.1 從 URDF 擷取的欄位

URDF 欄位	用途
joint parent / child	決定 Link 與 Joint 的父子關係
joint origin xyz / rpy	關節相對父 Link 的固定位置與方向
joint axis xyz	revolute joint 的局部旋轉軸
joint limit lower / upper	控制器與 IK 的角度邊界
visual mesh filename	Link 使用的 STL 視覺模型
visual origin xyz / rpy	STL 相對 Link frame 的安裝偏移

URDF 長度單位為公尺、角度為弧度；介面顯示時再轉成毫米與角度。

15.2 Three.js 父子階層

```
Scene / Base
├─ J1 fixedFrame → J1 rotor + Link 1
│   └─ J2 fixedFrame → J2 rotor + Link 2
│       └─ J3 fixedFrame → J3 rotor + Link 3
│           └─ J4 fixedFrame → J4 rotor + Link 4
│               └─ J5 fixedFrame → J5 rotor + Link 5
│                   └─ J6 fixedFrame → J6 rotor + Link 6 → TCP
```

fixedFrame 保存 URDF origin；rotor 只套用 joint axis 的旋轉。這使任何關節旋轉時，所有下游 Link 與 TCP 自動繼承變換。

15.3 目前 Joint frames

Joint	XYZ (m)	RPY (rad)	Axis
J1	0, 0, 0.092	$\pi, 0, 0$	0, 0, -1
J2	0, 0.06415, -0.07778	$\pi/2, 0, -\pi/2$	0, 0, -1
J3	0, -0.305, 0	0, 0, π	0, 0, -1
J4	0, 0, 0	$\pi/2, 0, -\pi/2$	0, 0, -1
J5	0, 0, -0.22294	$\pi, 0, -\pi/2$	1, 0, 0
J6	0, 0, 0.041	0, 0, 0	0, 0, 1

J1 使用 -Z axis，使從 +Z 上視時正角度為逆時針。

15.4 URDF RPY 轉換

Three.js 以 ZYX Euler order 套用 URDF roll、pitch、yaw：

```
object.rotation.set(roll, pitch, yaw, 'ZYX') R = Rz(yaw) × Ry(pitch) × Rx(roll)
```

若使用 Three.js 預設 XYZ order，J2、J4 等複合旋轉關節會產生錯誤的安裝面與旋轉軸。

15.5 STL Visual offsets

Link	STL XYZ offset (m)	STL RPY
Link 1	0, 0, 0	0, 0, 0
Link 2	0, 0, -0.00887	π , 0, 0
Link 3	0, 0, -0.03671	0, 0, 0
Link 4	0, 0, -0.07594	0, 0, $-\pi/2$
Link 5	0, 0, -0.0275	0, 0, 0
Link 6	0, 0, -0.016	0, 0, 0

Visual offset 只校正 STL 相對 Link frame 的顯示位置，不能拿來改變真正的 joint origin 或 axis。

15.6 ROS 2 資料與實機校正

URDF 提供 kinematic chain 的基準，但 STL 建模原點、控制器零點及期待的 Home 姿態可能不同，因此另設 JOINT_ZERO_OFFSETS。

套用角度 = UI 關節角度 + JOINT_ZERO_OFFSETS

目前只有 J1 使用 $+90^\circ$ 補償。J2 肩部幾何則透過 joint frame 與 visual transform 修正，不以零點偏移代替幾何修正。

15.7 共用同一套機械鏈

Three.js 場景、Forward Kinematics、TCP 座標、TCP 軸與數值 Inverse Kinematics 都使用同一套 joint hierarchy。畫面、位置計算與 IK 因此不會各自維護一套容易漂移的機械參數。